CS 330 Final Project: Design Decisions

Development Choices

For my final project, I created an interactive three-dimensional desk scene using C++ and OpenGL. I wanted the finished scene to resemble a realistic workspace while keeping the geometry simple enough to meet the low-polygon requirements. My scene includes a wooden desk, a speaker, a smartphone, a coffee mug, a computer monitor, and a potted plant. I arranged these objects using X, Y, and Z coordinates to give the workspace a balanced layout.

I constructed the objects from reusable primitive meshes rather than complex imported models. The speaker combines a cylinder and sphere; the coffee mug uses a cylinder and torus for its handle, and the monitor uses several boxes for its screen, stand, and base. The phone uses thin boxes, while the plant combines a tapered cylinder pot with overlapping spheres for foliage. The desk is a plane. Using these shapes helped me control scale, rotation, and placement while keeping the models relatively simple.

I applied image textures to the speaker's body, speaker front, and wooden desk to make the surfaces more recognizable. I also configured a white directional light and a soft pink point light. Ambient lighting keeps shaded surfaces visible; diffuse lighting shows the objects' shapes, and specular lighting adds highlights. I reduced the pink light intensity after testing, so the wood grain and object textures remained visible.

Camera Navigation and Interaction

I implemented camera controls so users can explore the scene from different angles. The W and S keys move forward and backward, A and D move left and right, and Q and E move the camera vertically. Moving the mouse changes the viewing direction, allowing the user to look around

without necessarily changing the camera position. Scrolling adjusts movement speed, with limits to prevent it from becoming too slow or too fast.

I also included two projection modes. Pressing O switches to orthographic projection, which removes perspective depth, while pressing P returns to perspective projection. These controls make it easier to inspect object placement and compare different views of the workspace. Testing the controls helped me confirm that the scene remained navigable as I added objects.

Code Organization and Reusability

I organized the program into functions with separate responsibilities. PrepareScene() loads the required textures, and primitive meshes and calls SetupSceneLights() to configure lighting. RenderScene() applies transformations, materials, textures, and drawing commands to each object. SetTransformations() allows me to reuse the same scaling, rotation, and translation process across different meshes instead of duplicating the underlying transformation logic.

The camera input functions also separate keyboard and mouse behavior from the scene rendering code. Descriptive comments identify the purpose of each object and lighting setting, making the project easier to review and update. Overall, this project strengthened my understanding of how basic geometry, texture mapping, Phong lighting, and camera interaction work together to create an interactive OpenGL scene.